Name: Student ID:

Agentic AI for Business and FinTech (FTEC5660)
Reproducibility Work, Due date: 13 November, midnight.

Instructions

- **Modality:** This is an individual homework. Students are encouraged to discuss with other students, but each student should submit their own work.
- **What you are doing:** Pick an agentic system project (code on GitHub), get it running, reproduce at least one reported result/behavior, then make one small, well-scoped modification and report what changed.

  This assignment is modeled after research-community reproducibility efforts: the goal is to verify what works, what does not, and why. This exercise is *not* about "pass/fail" a paper.

1 Choice of Project

You have two options:

1. Pick from the instructor-provided list; see below, or
2. Choose your own, as long as it is an agentic system and has enough documentation/results to evaluate.

Definition (for choosing repository to reproduce): an "agentic system" is a program that plans and acts over multiple steps (often with tools, memory, search, code execution, multi-agent loops, etc.), not just a single prompt-response.

Minimum eligibility checklist:

- Public GitHub repo with a runnable pipeline.
- A target to reproduce: a table/figure/metric, or at least a clear claimed capability with an evaluation script.
- You can run some meaningful subset within your compute/time budget.

2 Scope: you do not need to reproduce everything

Many agent papers are huge. You are allowed to reproduce one small task/experiment that is clearly defined (e.g., one dataset split, one benchmark, one ablation, one environment, one table row). This is normal in reproducibility work, you don't have to reproduce all experiments, especially if resource-heavy, and you can focus on a subset like baseline results.

Pick a "target claim" early. Example targets: "Table 2 accuracy on X", "success rate on Y tasks", "tool-call reliability", "cost per solved task", "latency", "win-rate vs baseline", etc.

3 You are not re-implementing from scratch

You do not need to re-build the method from the paper description. Using the authors' repo is explicitly acceptable in standard reproducibility challenges. Minimum expectation: run the repo (or a subset) and produce your own measured outputs, then compare them to what the paper/repo reports.

4 Edge case A: the repo uses an old/unsupported LLM → you may switch to DeepSeek

Many repos hard-code older models or deprecated endpoints. You may replace the model with DeepSeek (or another accessible model), as long as you document the change and treat it as a controlled variable. DeepSeek's API is designed to be OpenAI-compatible, so many repos can switch by changing config (model name + base URL + key) rather than rewriting the whole stack.

When you swap models:

- Record: model name, provider, base URL, decoding params (temperature/top_p/max_tokens), and any prompt/template changes.
- Be explicit: "Results are not directly comparable to the paper because the underlying LLM differs." Then compare as fairly as possible (same tasks, same metrics, same evaluation script).

5 Edge case B: the paper has many tasks → extract one small task and report

If the repo includes multiple benchmarks, pick one and go deeper:

- Reproduce the reported number(s) for that benchmark
- Identify hidden assumptions (dataset version, prompt format, seed, tool availability)
- Run 3–5 trials (if stochastic) and report mean/variance

This aligns with how reproducibility reports emphasize "what parts reproduce, at what cost, and under what conditions," rather than a binary outcome.

6 What counts as a "modification"?

Make one change that is small but meaningful, such as:

- Model change (e.g., to DeepSeek) with careful logging
- Prompt/tool policy change (e.g., stricter tool-use rules, different system prompt)
- Ablation (remove memory, remove planner step, disable a tool)
- Parameter change (max steps, reflection on/off, retrieval k, etc.)
- Evaluation tightening (fix a bug, pin dataset version, add seeds, add missing metric)

Your modification should be: (i) Isolated (one main change), (ii) Measurable (you can quantify impact).

7 Submission package (required)

Submit all three:

- Brief report (max 10 pages). Suggested structure:
  - Project summary + what you tried to reproduce

  - Setup notes (env, data, keys, compute)
  - Reproduction target(s) + metric definition
  - Results: your numbers vs reported numbers (tables/plots ok)
  - Your modification + results after modification
  - Debug diary: main blockers + how you resolved them
  - Conclusions: what is reproducible, what isn't, and why
- GitHub link that contains:
  - Clear README with install + run steps
  - Your changes in commits (no giant unreviewable dump)
  - No secrets/API keys committed
- Presentation link (maximal 15 minutes). A recorded Zoom presentation where you explain:
  - What you attempted to reproduce
  - Your modification and its measured impact
  - What worked / what didn't
  - Key lessons + recommendations to future users

8 Paper List by Theme

A Normally feasible with a laptop, hosted LLM API, and static or synthetic data.

B May require Docker, a local service/database, multimodal APIs, a larger number of calls, or careful use of a released checkpoint.

8.1 Planning, Search, and Self-Improvement

No.	Date	Full paper title and paper link	Code	Tier
1	2026-04	GenericAgent: A Token-Efficient Self-Evolving LLM Agent via Contextual Information Density Maximization	Repository	A
2	2026-03	AgentFactory: A Self-Evolving Framework Through Executable Subagent Accumulation and Reuse	Repository	A
3	2026-03	LiTS: A Modular Framework for LLM Tree Search	Repository	A
4	2025-05	Alita: Generalist Agent Enabling Scalable Agentic Reasoning with Minimal Predefinition and Maximal Self-Evolution	Repository	A
5	2025-02	QLASS: Boosting Language Agent Inference via Q-Guided Stepwise Search	Repository	B
6	2025-10	Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models	Repository	A
7	2026-05	R2V Agent: Teaching SLMs When to Ask for Help	Repository	B
8	2026-08	Real-Time Detection and Repair of LLM Agent Failures	Repository	A

8.2 Memory and Context Management

No.	Date	Full paper title and paper link	Code	Tier
9	2025-02	A-MEM: Agentic Memory for LLM Agents	Repository	A
10	2025-04	Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory	Repository	A
11	2025-06	Memory OS of AI Agent	Repository	A
12	2025-01	Zep: A Temporal Knowledge Graph Architecture for Agent Memory	Repository	A
13	2025-08	What Deserves Memory: Adaptive Memory Distillation for LLM Agents	Repository	A
14	2026-01	SimpleMem: Efficient Lifelong Memory for LLM Agents	Repository	A
15	2025-06	G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems	Repository	A
16	2025-07	MIRIX: Multi-Agent Memory System for LLM-Based Agents	Repository	B

8.3 Tool Use, Search, and Workflow Control

No.	Date	Full paper title and paper link	Code	Tier
17	2026-03	ToolTree: Efficient LLM Agent Tool Planning via Dual-Feedback Monte Carlo Tree Search and Bidirectional Pruning	Repository	A
18	2026-05	LLM Agents Already Know When to Call Tools – Even Without Reasoning	Repository	A
19	2026-04	InfoSeeker: A Scalable Hierarchical Parallel Agent Framework for Web Information Seeking	Repository	A
20	2026-06	CLI-Anything: Towards Agent-Native Computer Use	Repository	B
21	2026-07	LLM-as-a-Verifier: A General-Purpose Verification Framework	Repository	A

8.4 Application Systems, Governance, and Infrastructure

No.	Date	Full paper title and paper link	Code	Tier
22	2024-12	TradingAgents: Multi-Agents LLM Financial Trading Framework	Repository	A
23	2025-09	QuantAgent: Price-Driven Multi-Agent LLMs for High-Frequency Trading	Repository	A
24	2026-06	EvoDS: Self-Evolving Autonomous Data Science Agent with Skill Learning and Context Management	Repository	B
25	2026-03	ClinicalAgents: Multi-Agent Orchestration for Clinical Decision Making with Dual-Memory	Repository	A
26	2026-01	SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents	Repository	B
27	2026-02	Codified Context: Infrastructure for AI Agents in a Complex Codebase	Repository	A
28	2026-06	Harness-MU: A Safe, Governed, and Effective Harness for Multi-User LLM Agents	Repository	A
29	2026-01	Orchestral AI: A Framework for Agent Orchestration	Repository	A